IAI600 – Introduction to Applied Artificial Intelligence

Lab II: Model Training, Evaluation and Optimization

Main topic: Chapter 2 (Literature Book) – End-to-end Machine Learning project
Dataset: California Housing Dataset
Software: Python, Jupyter Notebook, Pandas, NumPy, Matplotlib, Scikit-learn
Disclaimer: This lab material has been prepared with the assistance of Generative AI and is based on Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow; please report any errors, inaccuracies, or limitations you identify.

Objective of the laboratory: The objective of this lab is to continue the end-to-end machine learning project started in Lab I. Students will train different regression models, evaluate and compare their performance, fine-tune the selected models, perform a final evaluation on unseen test data, and save the resulting machine learning model for future use.
In Laboratory I, the data was explored and prepared for machine learning. In this laboratory, we move from **data preparation** to **model development**.

Part I – Preparing the Data for Model Training

At the end of Lab I, you created training and test sets and developed a preprocessing pipeline. In this lab II, use the training data and preprocessing pipeline developed in Lab I.
Separate the target variable (`median_house_value`) from the input attributes:

```
housing = strat_train_set.drop(
    "median_house_value",
    axis=1
)

housing_labels = strat_train_set[
    "median_house_value"
].copy()
```

Apply the preprocessing pipeline:

```
housing_prepared = full_pipeline.fit_transform(
    housing
)
```

Inspect the shape of the prepared data:

```
housing_prepared.shape
```

Task 1.1 – Understand the Prepared Data

Answer the following questions:

1. What is the target variable?
2. What are the input features?
3. How many training observations are available?
4. How many features are available after preprocessing?
5. Why can the number of features be different from the original number of attributes?

Part II – Train a Linear Regression Model

The first model we will train is a Linear Regression model. Linear Regression attempts to learn a relationship between the input features and the target value.
Import and create the model:

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()

lin_reg.fit(
    housing_prepared,
    housing_labels
)
```

The model has now learned from the training data.

Task 2.1 – Train the Model

Train the Linear Regression model and confirm that the model has been fitted successfully. Consider the following questions:

1. What does the `fit()` operation do?
2. What information does the model learn from the training data?
3. Does the model "memorize" the house prices?

Part III – Generate and Interpret Predictions

Use the trained model to make predictions for a few observations from the training set.

```
some_data = housing.iloc[:5]
some_labels = housing_labels.iloc[:5]

some_data_prepared = full_pipeline.transform(
    some_data
)

predictions = lin_reg.predict(
    some_data_prepared
)

print("Predictions:", predictions)
print("Actual␣values:", list(some_labels))
```

Compare the predicted values with the actual house values.

Task 3.1 – Interpret the Predictions

Create a small table comparing: actual house value, predicted house value, and prediction error. Discuss:

1. Are the predictions close to the actual values?
2. Are some predictions substantially different?
3. What could cause the prediction errors?

Part IV – Evaluate the Linear Regression Model

A model should not be evaluated only by looking at a few predictions. One common regression metric is the **Root Mean Squared Error (RMSE)**.

RMSE is defined as:

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2}$$

Calculate the RMSE on the training data:

```
from sklearn.metrics import mean_squared_error
import numpy as np

housing_predictions = lin_reg.predict(
    housing_prepared
)

lin_rmse = np.sqrt(
    mean_squared_error(
        housing_labels,
        housing_predictions
    )
)

print("RMSE:", lin_rmse)
```

Task 4.1 – Interpret RMSE

Answer:

1. What does RMSE measure?
2. What are the units of RMSE in this problem?
3. Is a lower RMSE better or worse?
4. Relate the RMSE value to the typical house prices in the dataset.

Part V – Train a Decision Tree Regressor

Now train a Decision Tree Regressor using the same prepared training data.

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor(
    random_state=42
)

tree_reg.fit(
    housing_prepared,
    housing_labels
)
```

Generate predictions:

```
housing_predictions = tree_reg.predict(
    housing_prepared
)
```

Calculate the RMSE:

```
tree_rmse = np.sqrt(
    mean_squared_error(
        housing_labels,
        housing_predictions
    )
)

print("Decision␣Tree␣RMSE:", tree_rmse)
```

Task 5.1 – Compare Linear Regression and Decision Tree

Compare the RMSE of: Linear Regression vs Decision Tree Regression
Answer:

1. Which model has the lower training RMSE?
2. Does a lower training error necessarily mean that the model will perform better on unseen data?
3. What might happen if the Decision Tree is too complex?

Part VI – Investigate Overfitting

A model can achieve a very low error on the training data while performing poorly on unseen data. This is known as **overfitting**.
The Decision Tree model is particularly useful for demonstrating this concept. Use cross-validation to obtain a better estimate of the model's performance.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    tree_reg,
    housing_prepared,
    housing_labels,
    scoring="neg_mean_squared_error",
    cv=10
)

tree_rmse_scores = np.sqrt(-scores)
```

Calculate the mean and standard deviation:

```
print("Mean␣RMSE:", tree_rmse_scores.mean())
print("Standard␣deviation:",
     tree_rmse_scores.std())
```

Task 6.1 – Interpret Cross-Validation

Answer:

1. Why do we use multiple validation folds?
2. Why is the score returned by Scikit-learn negative?
3. What does the mean RMSE tell you?
4. What does the standard deviation tell you?
5. Why can cross-validation provide a more useful estimate than a single training score?

Part VII – Compare Different Models Using Cross-Validation

Use cross-validation to evaluate the Linear Regression model as well.

```
lin_scores = cross_val_score(
    lin_reg,
    housing_prepared,
    housing_labels,
    scoring="neg_mean_squared_error",
    cv=10
)

lin_rmse_scores = np.sqrt(-lin_scores)

print("Mean␣RMSE:",
      lin_rmse_scores.mean())

print("Standard␣deviation:",
      lin_rmse_scores.std())
```

Create a similar evaluation for a **Random Forest Regressor**:

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

forest_scores = cross_val_score(
    forest_reg,
    housing_prepared,
    housing_labels,
    scoring="neg_mean_squared_error",
    cv=10
)

forest_rmse_scores = np.sqrt(-forest_scores)

print("Mean␣RMSE:",
      forest_rmse_scores.mean())

print("Standard␣deviation:",
      forest_rmse_scores.std())
```

Task 7 – Compare the Models

Create a table similar to the following:

Model	Mean RMSE	Std. Dev.
Linear Regression		
Decision Tree		
Random Forest		

Answer:

1. Which model performs best according to the mean RMSE?
2. Which model gives the most consistent results?
3. Is the model with the lowest training error also the best model according to cross-validation?
4. What does this comparison tell you about evaluating machine learning models?

Part VIII – Fine-Tuning the Model

The performance of many machine learning models depends on their **hyperparameters**. Hyperparameters are settings that are selected before training rather than learned directly from the training data.
For the **Random Forest model**, examples include: number of trees (`n_estimators`) and maximum number of features considered at each split (`max_features`).
We can systematically search for a good combination of hyperparameters using `GridSearchCV`.

Grid Search

Create a parameter grid:

```
from sklearn.model_selection import GridSearchCV

param_grid = [
    {
        "n_estimators": [3, 10, 30],
        "max_features": [2, 4, 6, 8]
    },
    {
        "bootstrap": [False],
        "n_estimators": [3, 10],
        "max_features": [2, 3, 4]
    }
]
```

Create the grid search:

```
forest_reg = RandomForestRegressor(
    random_state=42
)

grid_search = GridSearchCV(
    forest_reg,
    param_grid,
    cv=5,
    scoring="neg_mean_squared_error",
```

```
    return_train_score=True
)

grid_search.fit(
    housing_prepared,
    housing_labels
)
```

Display the best parameters:

```
print(grid_search.best_params_)
```

Display the best estimator:

```
print(grid_search.best_estimator_)
```

Task 8.1 – Investigate the Grid Search

Answer:

1. Which combination of hyperparameters produced the best result?
2. How much did the performance vary between different combinations?
3. Why is systematic hyperparameter search useful?

Randomized Search

Grid search can become expensive when many hyperparameters and values are considered. An alternative is `RandomizedSearchCV`, which samples combinations from specified distributions or lists.
Create a randomized search for the Random Forest model.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distribs = {
    "n_estimators": randint(
        low=1,
        high=200
    ),
    "max_features": randint(
        low=1,
        high=8
    )
}

forest_reg = RandomForestRegressor(
    random_state=42
)

rnd_search = RandomizedSearchCV(
    forest_reg,
    param_distributions=param_distribs,
    n_iter=10,
    cv=5,
    scoring="neg_mean_squared_error",
    random_state=42
)
```

```
rnd_search.fit(
    housing_prepared,
    housing_labels
)
```

Display the best parameters:

```
print(rnd_search.best_params_)
```

Task 8.2 – Compare Grid Search and Randomized Search

Discuss:

1. What is the difference between Grid Search and Randomized Search?
2. Which method evaluates more combinations in this experiment?
3. When could Randomized Search be preferable?
4. Did Randomized Search find a better model in your experiment?

Part IX – Analyze the Best Model and Model Error

Analyze Best Model

After fine-tuning, examine the best Random Forest model.
Retrieve the feature importance:

```
feature_importances = (
    grid_search.best_estimator_
    .feature_importances_
)

print(feature_importances)
```

Task 9.1 – Interpret Feature Importance

Identify the most important features according to the selected Random Forest model.
Discuss:

1. Which features appear most important?
2. Does feature importance mean that a feature causes the target to change?
3. Are the results consistent with the correlation analysis performed in Lab I?
4. How could feature importance help an engineer or domain expert?

Analyze Model Errors

A good machine learning workflow does not stop after finding the model with the lowest validation error. We should investigate **where and why the model makes errors**.
Use the validation results and predictions from the best model to investigate the errors.
Calculate prediction errors:

```
final_model = grid_search.best_estimator_

predictions = final_model.predict(
    housing_prepared
)

errors = housing_labels - predictions

error_rmse = np.sqrt(
    np.mean(errors ** 2)
)

print("RMSE:", error_rmse)
```

Task 9.2 – Interpret Model Errors

Consider:

1. Are the errors similar for all observations?
2. Are there observations for which the model performs particularly poorly?

Part X – Final Evaluation on the Test Set

After selecting and fine-tuning the final model, evaluate it on the test set.
Important: The test set should not be used repeatedly during model development. It is intended to provide an approximately unbiased estimate of the final model's performance on unseen data.
Separate the test features and labels:

```
final_test = strat_test_set.drop(
    "median_house_value",
    axis=1
)

final_test_labels = strat_test_set[
    "median_house_value"
].copy()
```

Transform the test features using the already fitted preprocessing pipeline:

```
final_test_prepared = full_pipeline.transform(
    final_test
)
```

Generate the final predictions:

```
final_predictions = final_model.predict(
    final_test_prepared
)
```

Calculate the final RMSE:

```
final_rmse = np.sqrt(
    mean_squared_error(
        final_test_labels,
        final_predictions
    )
```

```
)

print("Final␣RMSE:", final_rmse)
```

You may also calculate the Mean Absolute Error (MAE):

```
from sklearn.metrics import mean_absolute_error

final_mae = mean_absolute_error(
    final_test_labels,
    final_predictions
)

print("Final␣MAE:", final_mae)
```

Task 10.1 – Interpret the Final Result

Answer:

1. What is the final RMSE?
2. What is the final MAE?
3. How do the final results compare with the cross-validation results?
4. How large is the typical prediction error relative to the house prices?

Part XI – Run a Complete Prediction Pipeline

A practical machine learning system should be able to take raw data and produce a prediction without requiring the user to manually repeat every preprocessing step.
Combine the preprocessing pipeline and trained model into one complete pipeline.

```
from sklearn.pipeline import Pipeline

full_prediction_pipeline = Pipeline([
    ("preparation", full_pipeline),
    ("model", final_model)
])
```

Use the complete pipeline to generate predictions from raw test data:

```
final_predictions = (
    full_prediction_pipeline.predict(
        final_test
    )
)
```

Task 11.1 – Understand the Complete Pipeline

Explain:

1. Which preprocessing steps are performed automatically?
2. Why is combining preprocessing and prediction useful?

Part XII – Save and Load the Trained Model (Optional)

A trained machine learning model can be saved and reused later without retraining it from the beginning. Please note that, this step is optional for our course IAI600. You will not be asked to demonstrate this part.
Use the `joblib` library:

```
import joblib

joblib.dump(
    full_prediction_pipeline,
    "california_housing_model.pkl"
)
```

Load the saved model:

```
loaded_model = joblib.load(
    "california_housing_model.pkl"
)
```

Use the loaded model to make predictions:

```
predictions = loaded_model.predict(
    final_test
)
```

Part XIII – Optional Extension: Support Vector Regression

This section is also optional. Support Vector Regression (SVR) can also be used for regression problems.
Try an SVR model using Scikit-learn:

```
from sklearn.svm import SVR

svm_reg = SVR(
    kernel="linear",
    C=1
)

svm_reg.fit(
    housing_prepared,
    housing_labels
)
```

Experiment with different values of `C` parameter.
You may also try the RBF kernel:

```
svm_reg = SVR(
    kernel="rbf",
    C=30,
    gamma="scale"
)

svm_reg.fit(
    housing_prepared,
    housing_labels
)
```

Evaluate the model using cross-validation and compare the result with the previous models.